

Thuật toán trong hệ thống thật (Algorithms in Production)

PHỤ THUỘC A/03 cho khoảng cách giữa tiệm cận và hằng số; D/24 cho các mô hình chi phí khác; E/25 cho kỷ luật đo đạc.

TÓM TẮT

Chương này nói về chỗ mà lý thuyết gặp thực tế, và về vì sao thuật toán tốt nhất trên giấy thường không phải thuật toán được chọn. Bốn chủ đề: hằng số và bộ nhớ có tiếng nói riêng ở kích thước thật; kỷ luật đo trước khi tối ưu, vì trực giác về nút cổ chai gần như luôn sai; các tiêu chí ngoài tốc độ — độ trễ đuôi, khả năng bảo trì, tính dự đoán được — vốn quyết định nhiều hơn trong hệ thống thật; và một khảo sát ngắn về nơi các thuật toán của quyển sách này đang thực sự chạy. Nếu chỉ nhớ một điều: trong hệ thống thật, câu hỏi không phải “thuật toán nào nhanh nhất” mà là “ràng buộc thật sự của tôi là gì” — và câu trả lời thường không phải số phép toán.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hồ sơ hiệu năng (profiling)** — đo xem thời gian thực sự bị tiêu ở đâu trong chương trình, thay vì đoán.
- **độ trễ đuôi** — thời gian phản hồi ở các phân vị cao — 99% hoặc 99,9% — thay vì trung bình; trong dịch vụ trực tuyến, đây mới là con số người dùng cảm nhận.
- **thông lượng so với độ trễ** — số việc làm được mỗi giây so với thời gian hoàn thành một việc; tối ưu cái này thường làm hại cái kia.
- **ngưỡng hoà vốn** — kích thước dữ liệu mà tại đó một thuật toán bậc thấp hơn bắt đầu thắng thuật toán có hằng số nhỏ hơn.
- **tối ưu hoá vi mô** — cải thiện hằng số mà không đổi thuật toán: đổi cấu trúc dữ liệu, giảm cấp phát, cải thiện kiểu truy cập bộ nhớ.
- **tính dự đoán được** — khả năng đảm bảo thời gian chạy cho *mỗi* lần gọi, không chỉ trung bình — yêu cầu đặc trưng của hệ thời gian thực.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao trực giác về “chỗ nào chậm” trong một chương trình lại gần như luôn sai?
- Q2. Khi nào một thuật toán có độ phức tạp tệ hơn lại là lựa chọn đúng trong sản phẩm thật?
- Q3. Vì sao độ trễ trung bình lại là con số gây hiểu lầm trong dịch vụ trực tuyến?
- Q4. Cùng một thuật toán, vì sao đổi cách sắp xếp dữ liệu trong bộ nhớ có thể làm nó nhanh gấp nhiều lần?
- Q5. Trong hệ thống thời gian thực, vì sao một cấu trúc có chi phí khấu hao rất tốt lại có thể là lựa chọn sai?

Có một khoảng cách mà người luyện thuật toán thường không thấy cho tới khi đi làm: trong kỳ thi, tiêu chí duy nhất là “đúng và trong giới hạn thời gian”. Trong hệ thống thật, có ít nhất năm tiêu chí cạnh tranh nhau — tốc độ trung bình, tốc độ ở trường hợp xấu, bộ nhớ, khả năng bảo

trì, và khả năng dự đoán — và thuật toán thắng cuộc là thuật toán cân bằng chúng, không phải thuật toán tối ưu một chiều.

Điều này không làm mất giá trị của phần A tới phần D. Ngược lại: bậc tiệm cận vẫn là thứ quyết định khi dữ liệu lớn lên, và không có nó thì bạn không có cách nào so sánh các hướng đi. Nhưng nó là *điều kiện cần chứ không đủ*, và chương này nói về phần còn thiếu.

1.1 Đo trước khi tối ưu

Đây là nguyên tắc quan trọng nhất chương, và lý do nó cần được nhắc lại là vì ai cũng đồng ý với nó mà rất ít người làm.

Câu của Knuth về tối ưu hoá sớm[1] thường bị trích cụt. Ý đầy đủ có hai vế: phần lớn thời gian ta nên bỏ qua các cải thiện nhỏ, *nhưng* không được bỏ lỡ cơ hội ở phần mã thực sự quan trọng — và cách biết phần nào quan trọng là *đo*, không phải đoán.

Vì sao trực giác sai? Ba lý do. Thứ nhất, chương trình hiện đại có nhiều tầng — thư viện, cấp phát bộ nhớ, hệ điều hành — và chi phí thường nằm ở tầng bạn không viết. Thứ hai, chi phí truy cập bộ nhớ không nhìn thấy được trong mã nguồn: hai vòng lặp trông giống hệt nhau có thể chênh nhau năm lần vì kiểu truy cập. Thứ ba, chúng ta có xu hướng chú ý tới đoạn mã *phức tạp* chứ không phải đoạn mã *chạy nhiều lần*, mà hai thứ đó thường không trùng nhau.

Quy trình tối thiểu: đo tổng thời gian trước; chạy hồ sơ hiệu năng để tìm phần chiếm nhiều nhất; chỉ tối ưu phần đó; đo lại để xác nhận. Bước cuối hay bị bỏ, và nó là bước quan trọng nhất — rất nhiều “tối ưu” hoá ra không cải thiện gì, hoặc làm chậm đi vì một hiệu ứng phụ.

1.2 Hằng số, bộ nhớ và những gì tiệm cận không nói

Ba nhóm yếu tố quyết định hiệu năng thật mà ký hiệu $O(\cdot)$ bỏ qua.

Kiểu truy cập bộ nhớ. Đây là yếu tố lớn nhất và bị đánh giá thấp nhất, đúng như hình phân cấp bộ nhớ ở chương A/03. Duyệt mảng tuần tự nhanh hơn nhiều lần so với duyệt danh sách liên kết cùng số phần tử; duyệt ma trận theo hàng nhanh hơn theo cột; cấu trúc cây cài bằng mảng nhanh hơn cài bằng con trỏ. Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó cũng là lý do “tối ưu vi mô” đầu tiên nên thử luôn là đổi bố cục dữ liệu chứ không phải đổi thuật toán.

Cấp phát bộ nhớ. Cấp phát động trong vòng lặp là một trong những nguyên nhân chậm phổ biến nhất và ít bị nghi ngờ nhất. Cấp phát trước, dùng lại vùng nhớ, và tránh tạo đối tượng tạm là những cải thiện thường cho hệ số lớn mà không đổi một dòng logic nào.

Ngưỡng hoà vốn. Mọi thuật toán “tốt hơn về tiệm cận” đều có một kích thước dưới đó nó thua. Các thư viện thật xử lý điều này bằng cách kết hợp: hàm sắp xếp chuẩn chuyển sang sắp xếp chèn khi đoạn nhỏ[2], và chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng[3]. Đây là câu trả lời cho câu hỏi mở đầu thứ hai ở dạng cụ thể nhất: thuật toán “tệ hơn” được chọn khi n nhỏ, và trong thực tế n thường nhỏ hơn ta tưởng.

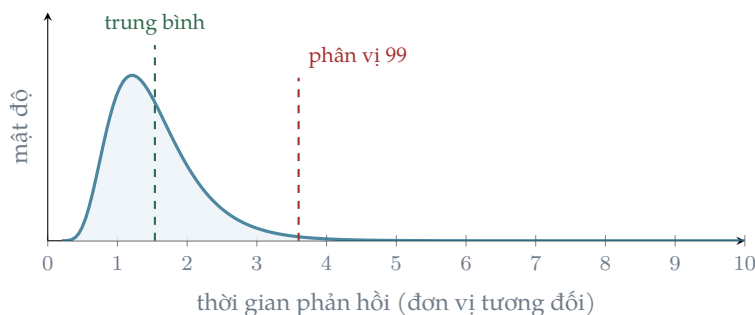
CẠM BẮY

Một cạm bẫy đặc thù của người mới đi làm sau khi luyện thi đấu: chọn cấu trúc dữ liệu tinh vi cho một bài mà dữ liệu chỉ có vài trăm phần tử. Một mảng phẳng và một vòng lặp tuyến tính thường vừa nhanh hơn vừa dễ bảo trì hơn một cây cân bằng tự cài. Câu hỏi đầu

tiên trong hệ thống thật không phải “cấu trúc nào tối ưu” mà là “dữ liệu lớn tới đâu, và nó sẽ lớn tới đâu trong hai năm nữa”.

1.3 Những tiêu chí ngoài tốc độ

Độ trễ đuôi. Trong một dịch vụ trực tuyến, một yêu cầu của người dùng thường phải chờ hàng chục thành phần con trả lời. Nếu mỗi thành phần chậm bất thường trong 1% số lần, thì xác suất *ít nhất một* thành phần chậm là rất cao — và người dùng cảm nhận đúng cái chậm nhất. Đó là lý do các hệ thống lớn đo và tối ưu theo phân vị 99 hoặc 99,9 thay vì theo trung bình. Hệ quả cho việc chọn thuật toán: một thuật toán có trung bình tốt nhưng đuôi xấu — ví dụ bảng băm phải băm lại toàn bộ khi vượt ngưỡng tải, hay bộ thu gom rác chạy đột ngột — có thể tệ hơn một thuật toán chậm hơn nhưng đều đặn.



Hình 1.1. Minh họa bằng phân phối log-normal với $\mu = 0.35, \sigma = 0.4$, không phải số đo hệ thống thật. Trung bình xấp xỉ 1.537 và phân vị 99 xấp xỉ 3.599: phần lớn yêu cầu nhanh, một số ít rất chậm. Khi một thao tác của người dùng chạm tới hàng chục thành phần, họ cảm nhận cái chậm nhất — nên phân vị 99 mới là con số phản ánh trải nghiệm, và tính *đều đặn* của thuật toán trở nên quan trọng ngang tốc độ trung bình.

Tính dự đoán được. Trong hệ thời gian thực — điều khiển robot, xử lý âm thanh, hệ thống an toàn — yêu cầu là *mỗi* chu kỳ phải hoàn thành trước hạn. Ở đó, chi phí khấu hao đẹp không cứu được bạn: một thao tác đột ngột tốn $O(n)$ làm lỡ hạn kỳ và hậu quả có thể là vật lý. Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó dẫn tới các lựa chọn khác hẳn: cấu trúc có đảm bảo trường hợp xấu nhất, cấp phát tĩnh thay vì động, và tránh các cơ chế chạy nền không kiểm soát được thời điểm.

Khả năng bảo trì. Một thuật toán mà chỉ một người trong nhóm hiểu là một rủi ro vận hành. Trong thực tế, chênh lệch 20% tốc độ hiếm khi biện minh được cho việc thay một lời giải rõ ràng bằng một lời giải tinh vi. Nguyên tắc thực dụng: dùng cấu trúc phức tạp khi nó cho cải thiện *bậc*, không phải khi nó cho cải thiện hằng số.

Tính đúng dưới dữ liệu thù địch. Đây là chỗ chương B/08 và C/20 chạm vào hệ thống thật: nếu dữ liệu vào đến từ người dùng bên ngoài, phân tích trung bình không còn đủ. Bảng băm không ngẫu nhiên hoá, biểu thức chính quy có thể bị đưa vào trạng thái mũ, và các thuật toán sắp xếp có trường hợp xấu đoán được — cả ba đều đã từng bị khai thác để làm quá tải dịch vụ.

1.4 Thuật toán của quyển sách này đang chạy ở đâu

Mục này là khảo sát ngắn, đọc để có bức tranh chứ không cần thuộc.

Cơ sở dữ liệu là nơi tập trung nhiều thuật toán nhất: B-cây cho chỉ mục (B/09), sắp xếp ngoài và trộn nhiều đường (D/24), nối bằng băm và nối bằng trộn, và một bộ tối ưu truy vấn về bản chất là bài toán tìm kiếm trên không gian kế hoạch thực thi.

Trình biên dịch dùng đồ thị ở khắp nơi (C/17): đồ thị luồng điều khiển, sắp thứ tự tô-pô để xếp lịch lệnh, tô màu đồ thị để cấp phát thanh ghi — một bài NP-khó được giải bằng heuristic trong mọi lần biên dịch.

Mạng và hạ tầng dùng Dijkstra và Bellman–Ford ở dạng phân tán cho định tuyến, băm nhất quán cho phân mảnh dữ liệu (B/08), và các cấu trúc xấp xỉ cho giám sát lưu lượng (D/24).

Đồ hoạ và trò chơi dùng cấu trúc phân cấp không gian cho dò tia và va chạm (D/22), A* cho tìm đường, và rất nhiều tối ưu hằng số vì ràng buộc thời gian khung hình rất chặt.

Robot và hệ thống nhận thức dùng bài toán tối ưu trên đồ thị làm khung chính — đồ thị tư thế trong định vị và dựng bản đồ — cộng với cây k -d cho tìm lân cận, RANSAC cho loại ngoại lai (C/20), và A* cho lập kế hoạch. Điểm đáng chú ý: ở đây ràng buộc thời gian thực và ràng buộc năng lượng thường quyết định nhiều hơn bậc tiệm cận, đúng như mục 3 nói.

Học máy dùng đại số tuyến tính và tối ưu liên tục làm lõi, nhưng các thuật toán rời rạc của quyển sách này vẫn có mặt ở khắp phần hạ tầng: lấy mẫu, chỉ mục vector cho tìm lân cận gần đúng, cấu trúc dữ liệu cho tải dữ liệu, và các bài chọn tập con có tính dưới mô-đun (C/15).

1.5 Một phiếu đo trước khi quyết định tối ưu

Với một pipeline xử lý khung hình, hãy đo riêng đọc dữ liệu, tiền xử lý, thuật toán lõi và xuất kết quả, rồi đo lại toàn pipeline. Giả sử phép tìm lân cận chiếm 30% tổng thời gian: tăng tốc phần này hai lần chỉ cho hệ số toàn cục

$$\frac{1}{(1 - 0.3) + 0.3/2} \approx 1.18.$$

Ngay cả làm phần đó nhanh vô hạn cũng chỉ đạt $1/0.7 \approx 1.43$ lần. Đây là một phép tính từ tỉ phần thời gian giả định, không phải số đo benchmark.

Bảng 1.1. Phiếu đo để so sánh ba thay đổi thường gặp. Luôn dùng cùng tập đầu vào và kiểm kết quả trước khi so tốc độ.

Thay đổi	Dự đoán cần kiểm	Đo thêm	Đánh đổi
Mảng phẳng thay nút cấp phát riêng	Giảm cấp phát và cache miss	Bộ nhớ đỉnh, thời gian xây	Chi phí chèn/xoá, di chuyển dữ liệu
Tiền xử lý chỉ mục	Truy vấn rẻ hơn sau khi trả chi phí xây	Tổng thời gian theo số truy vấn q	Chỉ mục mất hiệu lực khi dữ liệu đổi
Xấp xỉ tìm lân cận	Đổi chất lượng lấy độ trễ thấp hơn	Recall so với tìm chính xác, p95/p99	Sai khác có thể ảnh hưởng bước tối ưu phía sau

Bảng 1.1 buộc mỗi tối ưu có một dự đoán có thể bác bỏ. Ghi kích thước và phân phối dữ liệu, phiên bản mã, chế độ biên dịch, số luồng, phân cứng và số lần lặp. Tách phép đo cache nóng/lạnh; chạy xen kẽ bản cũ và mới để giảm sai lệch do nhiệt độ hoặc tải nền. Với robot, kiểm cả tỉ lệ trễ hạn trên toàn pipeline: p99 thấp không đồng nghĩa mọi chu kỳ đều kịp hạn.

1.6 Gốc rễ: từ “đừng tối ưu sớm” tới “đuôi mới là thứ quan trọng”

Câu chuyện về mối quan hệ giữa lý thuyết và thực hành trong ngành này đi qua vài giai đoạn khá rõ.

Thập niên 1970 là giai đoạn mà tối ưu hoá thủ công còn phổ biến và thường phản tác dụng. Knuth viết câu nổi tiếng về tối ưu hoá sớm trong bối cảnh đó[1], và Bentley viết cả một quyển sách về việc viết chương trình hiệu quả[4] với thông điệp cân bằng: có kỹ thuật thật, nhưng phải áp dụng ở đúng chỗ tìm được bằng đo đạc.

Thập niên 1990 mang tới một bài học khác từ việc kỹ thuật hoá các thư viện chuẩn. Bài viết của Bentley và McIlroy về hàm sắp xếp[2] là tài liệu đáng đọc vì nó cho thấy khoảng cách giữa thuật toán trong sách và hàm dùng được: phải xử lý dữ liệu đã sắp sẵn, dữ liệu có nhiều phần tử bằng nhau, dữ liệu thù địch, và phải chuyển thuật toán theo kích thước. Cùng thời kỳ, introsort[3] giải quyết vấn đề trường hợp xấu bằng cách kết hợp ba thuật toán.

Từ giữa thập niên 2000, khi dịch vụ trực tuyến quy mô lớn trở nên phổ biến, một mối quan tâm mới nổi lên: *độ trễ đuôi*. Nhận thức rằng trung bình là con số gây hiểu lầm khi một yêu cầu phụ thuộc vào hàng chục thành phần đã thay đổi cách các hệ thống lớn được thiết kế và đo lường — và nó cũng thay đổi tiêu chí chọn thuật toán, đưa tính đều đặn lên ngang hàng với tốc độ trung bình.

Cuối cùng, bài báo về chi phí của khả năng mở rộng[5] là một lời nhắc có tính hệ thống: nhiều hệ thống xử lý dữ liệu song song, khi đo công bằng, thua một chương trình đơn luồng viết cẩn thận. Thông điệp không phải “đừng song song hoá” mà là “hãy đo so với một mốc trung thực” — và mốc đó thường là một cài đặt đơn giản, tốt, chạy trên một máy.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1974 — Knuth	Tối ưu hoá thủ công tràn lan làm mã khó đọc mà không nhanh hơn.	“Đừng tối ưu sớm” — nhưng về sau bị quên: phải đo để tìm phần thực sự quan trọng và tối ưu ở đó.
1982 — Bentley	Cần một cách có hệ thống để tăng tốc chương trình.	Danh mục kỹ thuật kèm nguyên tắc: chỉ áp dụng ở nơi đo đạc chỉ ra.
1993 — Bentley & McIlroy	Hàm sắp xếp trong thư viện chuẩn hỏng trên dữ liệu thật.	Kỹ thuật hoá: xử lý dữ liệu đã sắp, nhiều phần tử bằng nhau, và chuyển thuật toán theo kích thước.
1997 — Musser	Quicksort có trường hợp xấu $O(n^2)$ khai thác được.	Introsort: chuyển sang heapsort khi đệ quy quá sâu — kết hợp ba thuật toán để có cả tốc độ lẫn đảm bảo.
Giữa thập niên 2000 — dịch vụ quy mô lớn	Một yêu cầu phụ thuộc hàng chục thành phần; trung bình tốt mà người dùng vẫn thấy chậm.	Độ trễ đuôi trở thành tiêu chí chính; tính đều đặn được coi trọng ngang tốc độ trung bình.
2015 — McSherry và cộng sự	Hệ thống phân tán được đánh giá bằng khả năng mở rộng chứ không bằng tốc độ tuyệt đối.	So với mốc trung thực — một cài đặt đơn luồng tốt — nhiều hệ thống song song thua.

I 1.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kỹ thuật hệ thống	Đo tải, phân vị độ trễ, ngân sách lỗi	Cùng nguyên tắc “đo trước khi quyết”. Điểm khác so với thi đấu: mục tiêu không phải giá trị nhỏ nhất mà là <i>đạt một ngưỡng cam kết một cách ổn định</i> .
Sản xuất công nghiệp	Lý thuyết nút cổ chai: cải thiện chỗ không phải nút thắt thì không tăng sản lượng	Trùng khít với nguyên tắc hồ sơ hiệu năng ở mục 1. Bài học chung: tối ưu ngoài nút cổ chai vừa vô ích vừa che mắt vấn đề thật.
Kinh tế học	Chi phí cơ hội, lợi ích cận biên giảm dần	Mỗi giờ tối ưu hằng số là một giờ không dùng cho việc khác. Trong hệ thống thật, “đủ nhanh” là một trạng thái hợp lệ, khác hẳn tinh thần thi đấu.
Hệ thời gian thực và robot	Ràng buộc hạn kỳ cứng, phân tích thời gian chạy xấu nhất	Ở đây tiêu chí là <i>trường hợp xấu nhất mỗi chu kỳ</i> , nên các cấu trúc có khâu hao đẹp bị loại. Đây là mô hình chi phí riêng mà D/24 đã nhắc.
Kiến trúc máy tính	Bộ nhớ đệm, dự đoán rẽ nhánh, xử lý theo lô	Giải thích vì sao kiểu truy cập bộ nhớ quan trọng hơn số phép toán ở kích thước thật. Hiểu sơ bộ tăng này biến “tối ưu vi mô” từ mơ mẫm thành có định hướng.
Hàng không, y tế	Thiết kế cho trường hợp xấu, không cho trường hợp trung bình	Cùng tinh thần với tính dự đoán được ở mục 3: trong hệ thống mà hậu quả nghiêm trọng, đảm bảo quan trọng hơn hiệu năng trung bình.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì ba lý do. Chi phí thường nằm ở tầng bạn không viết — thư viện, cấp phát bộ nhớ, hệ điều hành. Chi phí truy cập bộ nhớ không nhìn thấy được trong mã nguồn, nên hai vòng lặp giống hệt nhau có thể chênh nhiều lần. Và chúng ta chú ý tới đoạn mã *phức tạp* thay vì đoạn mã *chạy nhiều lần*, mà hai thứ đó thường khác nhau. Đó là lý do quy trình bắt buộc phải là đo, tối ưu, rồi đo lại — và bước đo lại là bước hay bị bỏ nhất.
- A2. Ít nhất bốn tình huống. Khi n luôn nhỏ và nằm dưới ngưỡng hoà vốn — rất phổ biến hơn ta tưởng. Khi thuật toán “tệ hơn” có kiểu truy cập bộ nhớ tốt hơn và thắng trên máy thật. Khi nó đơn giản hơn nhiều và cả nhóm hiểu được, trong khi lợi ích chỉ là hằng số. Và khi nó có *đảm bảo trường hợp xấu nhất* tốt hơn, dù trung bình tệ hơn — điều quyết định trong hệ có hạn kỳ hoặc có dữ liệu thù địch.
- A3. Vì một yêu cầu của người dùng thường phụ thuộc vào nhiều thành phần con, và người dùng cảm nhận cái chậm nhất chứ không phải cái trung bình. Nếu mỗi thành phần chậm bất thường trong 1% số lần và một yêu cầu chậm tới vài chục thành phần thì xác suất gặp ít nhất một lần chậm là rất lớn. Vì vậy các hệ thống lớn đo theo phân vị cao, và tiêu chí chọn thuật toán chuyển từ “nhanh trung bình” sang “đều đặn”.
- A4. Vì đơn vị chi phí thật trên máy hiện đại là *lần chạm bộ nhớ chậm*, không phải phép toán. Dữ liệu nằm liên tiếp cho phép phần cứng nạp sẵn phần tiếp theo và tận dụng trọn vẹn mỗi dòng cache; dữ liệu nằm rải rác làm mỗi phần tử thành một lần chờ. Chênh lệch giữa hai trường hợp có thể tới hàng chục lần mà số phép toán không đổi — đúng như mô hình ở D/24 mô tả, chỉ ở quy mô cache thay vì đĩa.
- A5. Vì khâu hao chỉ *đảm bảo tổng* chi phí của một dãy thao tác, không đảm bảo từng thao tác. Trong hệ thời gian thực, một lần cấp phát lại toàn bộ hoặc một lần bấm lại bảng có thể tốn $O(n)$ đúng vào chu kỳ đang có hạn kỳ, và hậu quả là lỡ hạn — trong điều khiển robot thì đó là hậu quả vật lý. Ở đó người ta chọn cấu trúc có *đảm bảo trường hợp xấu nhất* cho từng thao tác, chấp nhận hằng số lớn hơn và thông lượng thấp hơn.

NÓI LẠI THẬT ĐƠN GIẢN

Trong kỳ thi, chỉ có một câu hỏi: chương trình có chạy đúng trong giới hạn thời gian không. Ngoài đời có thêm bốn câu nữa, và chúng thường quan trọng hơn.

Câu thứ nhất: chậm ở đâu? Câu trả lời gần như luôn khác chỗ bạn đoán. Người ta hay đi tối ưu đoạn mã trông rắc rối nhất, trong khi thời gian thật lại bị tiêu ở một vòng lặp tầm thường chạy hàng

triệu lần, hoặc ở việc cấp phát bộ nhớ mà bạn không hề viết ra. Cách duy nhất để biết là đo — và đo lại sau khi sửa, vì rất nhiều “cải tiến” hoá ra không cải tiến gì.

Câu thứ hai: dữ liệu thật lớn tới đâu? Nếu chỉ có vài trăm phần tử thì một vòng lặp đơn giản gần như luôn thắng một cấu trúc tinh vi, cả về tốc độ lẫn về việc người khác trong nhóm hiểu được nó.

Câu thứ ba: bạn quan tâm tới trung bình hay tới lần chậm nhất? Một dịch vụ có thời gian phản hồi trung bình rất đẹp vẫn có thể làm người dùng bức, vì mỗi thao tác của họ đọng tới hàng chục thành phần và họ cảm nhận cái chậm nhất. Còn trong điều khiển robot thì chỉ có lần chậm nhất là quan trọng: trung bình đẹp mà một chu kỳ lỡ hạn thì hỏng việc.

Câu thứ tư: ai sẽ sửa cái này sau một năm? Một lời giải nhanh hơn 20% nhưng chỉ một người hiểu thường là lựa chọn tệ hơn một lời giải rõ ràng.

Cuối cùng, một điều rất thực dụng mà lý thuyết không nói: trên máy hiện đại, việc dữ liệu nằm liền nhau trong bộ nhớ hay nằm rải rác có thể tạo chênh lệch nhiều lần, dù số phép tính không đổi. Vì vậy khi cần tăng tốc, thứ đầu tiên nên thử thường không phải đổi thuật toán mà là đổi cách xếp dữ liệu.

Một câu: Trong hệ thống thật, câu hỏi không phải “thuật toán nào nhanh nhất” mà “ràng buộc thật sự của tôi là gì”.

Chỗ không rút gọn được: Việc cân bằng giữa tốc độ, độ phức tạp mã và khả năng bảo trì không có công thức; nó là một quyết định kỹ thuật phụ thuộc bối cảnh và con người.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Nguyên tắc số một	Đo trước khi tối ưu, và <i>đo lại</i> sau khi tối ưu. Trực giác về nút cổ chai gần như luôn sai.
●●● Ba lý do trực giác sai	Chi phí nằm ở tầng bạn không viết; chi phí bộ nhớ không thấy trong mã nguồn; ta chú ý mã <i>phức tạp</i> thay vì mã <i>chạy nhiều</i> .
●●● Yếu tố hiệu năng lớn nhất mà tiệm cận bỏ qua	Kiểu truy cập bộ nhớ. Đảo bố cục dữ liệu thường cho hệ số lớn hơn đối thuật toán, ở kích thước thật.
●●● Khi nào chọn thuật toán “tệ hơn”	n luôn nhỏ; kiểu truy cập bộ nhớ tốt hơn; đơn giản hơn nhiều mà lợi ích chỉ là hằng số; hoặc có đảm bảo xấu nhất tốt hơn.
●●● Độ trễ đuôi	Người dùng cảm nhận thành phần chậm nhất, không phải trung bình; hệ thống lớn đo phân vị 99 hoặc 99,9. Tính đều đặn ngang hàng tốc độ.
●●● Hệ thời gian thực	Cần đảm bảo cho <i>mỗi</i> thao tác; khấu hao không đủ. Ưu tiên cấu trúc cố cận trên xấp xỉ, cấp phát tĩnh, tránh cơ chế chạy nền.
●● Ngưỡng hoà vốn	Mọi thuật toán tốt hơn về tiệm cận đều thua ở kích thước nhỏ; thư viện thật giải quyết bằng cách kết hợp nhiều thuật toán theo kích thước và độ sâu.
●● Cấp phát bộ nhớ	Cấp phát trong vòng lặp là nguyên nhân chậm phổ biến và ít bị nghi ngờ; cấp phát trước và dùng lại vùng nhớ thường cho cải thiện lớn.
●● Dữ liệu thù địch	Khi dữ liệu đến từ bên ngoài, phân tích trung bình không đủ: bảng băm phải ngẫu nhiên hoá, và các trường hợp xấu đoán được đều là lỗ hổng.
●● Khả năng bảo trì	Dùng cấu trúc phức tạp khi nó cho cải thiện <i>bậc</i> , không phải khi cho cải thiện hằng số.
●● Ý đầy đủ của Knuth	Bỏ qua cải thiện nhỏ ở phần lớn mã, nhưng <i>không</i> bỏ lỡ ở phần thực sự quan trọng — và phải đo để biết phần nào.
● Bài học COST	So sánh với một mốc trung thực — một cài đặt đơn luồng tốt — trước khi kết luận rằng hệ thống song song nhanh hơn.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên bắt đầu tối ưu từ đâu khi chương trình chậm?

| Theo thứ tự này. Một, đo để biết phần nào chiếm thời gian — đừng bỏ qua bước này dù bạn “chắc chắn” biết. Hai, kiểm tra bậc tiệm cận của phần đó: nếu sai bậc thì mọi tối ưu vi mô đều vô ích. Ba, nếu bậc đúng, xem kiểu truy cập bộ nhớ và cấp phát — hai thứ này thường cho hệ số lớn nhất. Bốn, chỉ sau đó mới tới các tối ưu vi mô khác. Và luôn đo lại sau mỗi thay đổi.

2. Làm sao biết mình đang bị chặn bởi tính toán hay bởi bộ nhớ?

| Hai thí nghiệm rẻ đã nói ở D/24: giảm kích thước dữ liệu xuống mức vừa bộ nhớ đệm và xem thời gian trên mỗi phần tử có giảm mạnh không; và đổi thứ tự truy cập mà giữ nguyên số phép toán. Nếu thời gian đổi nhiều lần trong thí nghiệm thứ hai thì bạn đang bị chặn bởi bộ nhớ, và việc cần làm là đổi bố cục dữ liệu chứ không phải giảm số phép tính.

3. Trong công việc, tôi có bao giờ cần tới các thuật toán ở phần C và D không?

| Có, nhưng không đều. Những thứ dùng gần như hằng ngày: sắp xếp và tìm kiếm, bảng băm, đồ thị cơ bản, quy hoạch động ở dạng đơn giản, nhị phân trên đáp án. Những thứ dùng vài lần trong sự nghiệp nhưng khi cần thì rất đáng: luồng cực đại cho các bài phân công, cấu trúc chuỗi cho tìm kiếm văn bản, nhận diện NP-khó để ngừng đúng lúc. Giá trị lớn nhất của phần C và D không phải là bạn cài chúng thường xuyên, mà là bạn *nhận ra* khi một bài trong công việc thuộc về một trong các mẫu đó.

4. Nên tự cài hay dùng thư viện?

| Mặc định là dùng thư viện: nó đã được kỹ thuật hoá, kiểm thử, và người khác đọc được. Tự cài khi có lý do cụ thể: thư viện thiếu thao tác bạn cần (tách/ghép cây, truy vấn thứ hạng), hằng số của nó là nút cổ chai đã đo được, hoặc bạn cần đảm bảo trường hợp xấu nhất mà thư viện không cho. Trong cả ba trường hợp, hãy stress test bản tự cài với thư viện làm trọng tài (E/25).

5. “Đủ nhanh” nghĩa là gì trong thực tế?

| Nghĩa là đạt một ngưỡng đã thoả thuận, ổn định, với biên an toàn cho tăng trưởng. Cách làm đúng là xác định ngưỡng *trước*: thời gian phản hồi mục tiêu ở phân vị 99, hoặc thời gian xử lý mỗi khung hình, hoặc dung lượng bộ nhớ tối đa. Không có ngưỡng thì việc tối ưu không có điểm dừng, và mọi giờ bỏ vào đó là một giờ không dùng cho việc khác — đây là nơi tinh thần kỹ thuật khác hẳn tinh thần thi đấu.

6. Người luyện thi đấu chuyển sang làm sản phẩm hay mắc lỗi gì?

| Ba lỗi. Chọn cấu trúc tinh vi cho dữ liệu nhỏ, làm mã khó bảo trì mà không nhanh hơn. Tối ưu hằng số trước khi kiểm tra bậc, hoặc ngược lại, đổi thuật toán khi nút cổ chai nằm ở nơi khác hoàn toàn. Và bỏ qua các tiêu chí ngoài tốc độ — đặc biệt là tính đều đặn và khả năng đọc hiểu. Điểm tốt mà họ mang sang: kỷ luật ước lượng chi phí trước khi cài, và thói quen kiểm thử bằng dữ liệu biên.

7. Có cách nào giữ được kỹ năng thuật toán khi công việc ít dùng tới?

| Ba cách rẻ. Duy trì một nhịp luyện nhỏ nhưng đều — một bài mỗi tuần đủ để giữ phản xạ (E/28). Chủ động tìm chỗ áp dụng trong công việc: rất nhiều bài “tối ưu lịch”, “chọn tập đại diện”, “tìm phụ thuộc vòng” là bài trong quyển sách này khoác áo nghiệp vụ. Và đọc mã nguồn của các thư viện bạn dùng — đó là nơi lý thuyết và kỹ thuật gặp nhau, và là tài liệu học tốt nhất về nội dung của chương này.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Đo thời gian duyệt mảng theo hàng và theo cột trên ma trận lớn	Bài tập thực nghiệm	Cùng số phép toán, khác kiểu truy cập; ghi lại tỉ số trên máy của bạn.
● Chạy hồ sơ hiệu năng cho một chương trình bạn đã viết	Bài tập công cụ	So kết quả với dự đoán của chính bạn trước khi đo — thường rất khác.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Tìm ngưỡng hoà vốn giữa sắp xếp chèn và sắp xếp trộn	Bài tập thực nghiệm	Vẽ đường thời gian theo n ; so con số của bạn với ngưỡng mà thư viện chuẩn dùng.
•• So sánh danh sách liên kết và mảng động cho cùng khối lượng công việc	Bài tập thực nghiệm	Đo chèn, duyệt, xoá; giải thích kết quả bằng phân cấp bộ nhớ chứ không bằng độ phức tạp.
•• Loại bỏ cấp phát trong vòng lặp nóng và đo lại	Bài tập tối ưu	Cấp phát trước và dùng lại vùng nhớ; đây thường là tối ưu có tỉ lệ lợi ích trên công sức cao nhất.
•• Đo phân vị 50, 99 và 99,9 của thời gian phản hồi thay vì trung bình	Bài tập đo lường	Dùng một dịch vụ nhỏ tự viết; quan sát khoảng cách giữa trung bình và đuôi.
••• Dựng dữ liệu thù địch cho một cấu trúc băm không ngẫu nhiên hoá	B/08	Đo mức suy giảm hiệu năng, rồi sửa bằng hạt giống ngẫu nhiên và đo lại.
••• So một chương trình đơn luồng viết tốt với một phiên bản song song	[5]	Chọn một bài xử lý đồ thị vừa phải; báo cáo cả thời gian lẫn số lỗi dùng.
••• Thay một cấu trúc dựa trên con trỏ bằng mảng phẳng trong một chương trình có sẵn	Bài tập tối ưu	Đo trước và sau; ghi lại cả thời gian lẫn mức tiêu thụ bộ nhớ.
••• Chọn một thuật toán trong quyển sách này và tìm nơi nó chạy trong một phần mềm mã nguồn mở bạn dùng	Bài tập đọc mã	Đọc cách nó được kỹ thuật hoá so với bản trong sách; đây là bài tập học được nhiều nhất chương.

Kết chương và đọc tiếp

Chương này nối phần lý thuyết với thực tế bằng một thay đổi trong câu hỏi: không phải “thuật toán nào nhanh nhất” mà “ràng buộc thật sự của tôi là gì”. Ba điều đáng mang theo: đo trước và đo lại sau khi tối ưu; kiểu truy cập bộ nhớ thường quyết định nhiều hơn số phép toán ở kích thước thật; và các tiêu chí ngoài tốc độ — đuôi độ trễ, tính dự đoán được, khả năng bảo trì — thường là thứ chọn ra thuật toán được dùng.

Hai chương cuối quay lại với người học. Chương E/27 nói về chiến thuật khi có giới hạn thời gian và có người quan sát — phỏng vấn và thi đấu — nơi kỹ năng quan trọng nhất không phải kiến thức mà là quản lý thời gian và nói ra suy nghĩ. Sau đó, chương E/28 biến toàn bộ quyển sách thành một lịch luyện cụ thể kèm ngân hàng bài tập.

Nguồn

- [1] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [2] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [3] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [4] J. Bentley, *Writing Efficient Programs*. Prentice-Hall, 1982.
- [5] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.